Solution: Construct Binary Tree from Preorder and Inorder Traversal

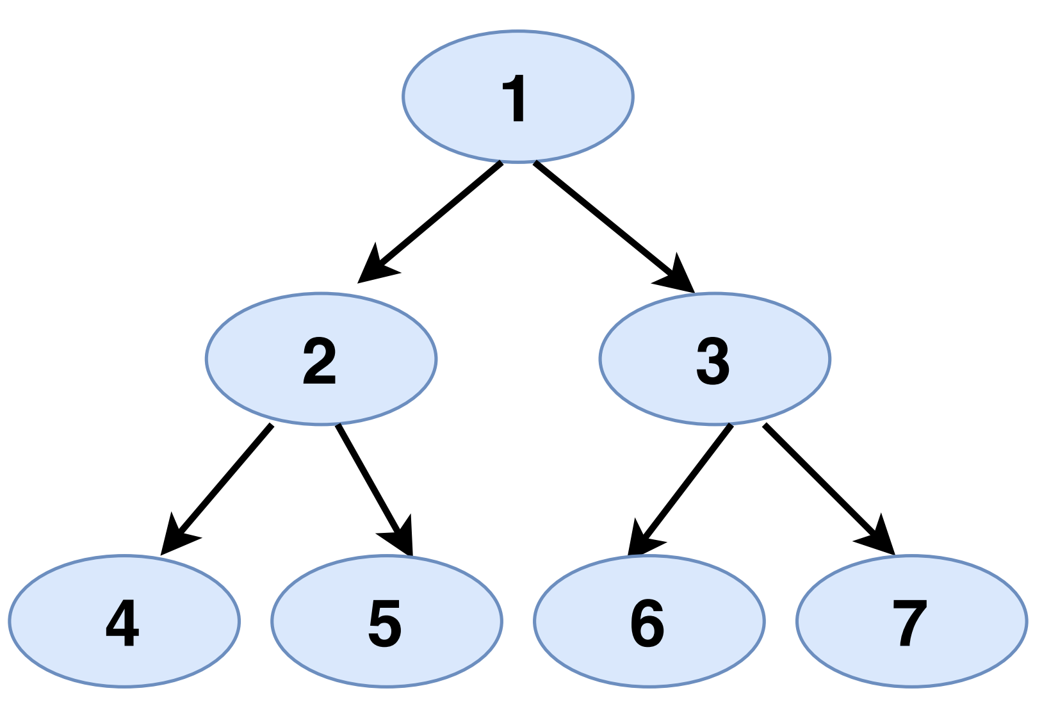
Problem Statement

Given the preorder and inorder traversal sequences of a binary tree, your task is to reconstruct this binary tree. Assume that the tree does not contain duplicate values.

Example Generation

Example 1:

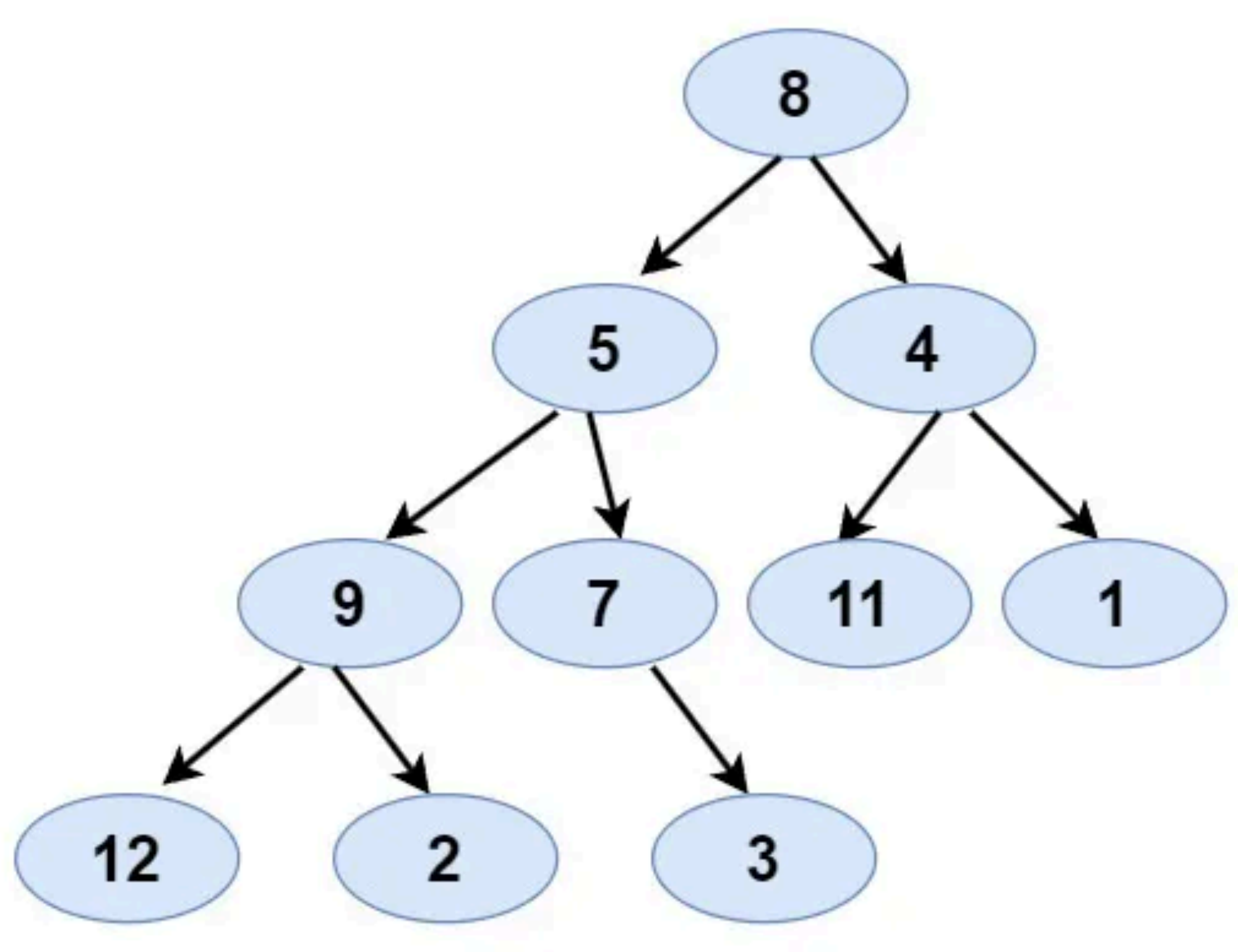
- Input:
  - Preorder: [1,2,4,5,3,6,7]
  - Inorder: [4,2,5,1,6,3,7]
- Expected Output:
  - Tree Representation: [1,2,3,4,5,6,7]



- Justification:
  - The first value in preorder (1) is the root. In the inorder list, everything left of value 1 is the left subtree and everything on the right is the right subtree. Following this pattern recursively helps in reconstructing the binary tree.

Example 2:

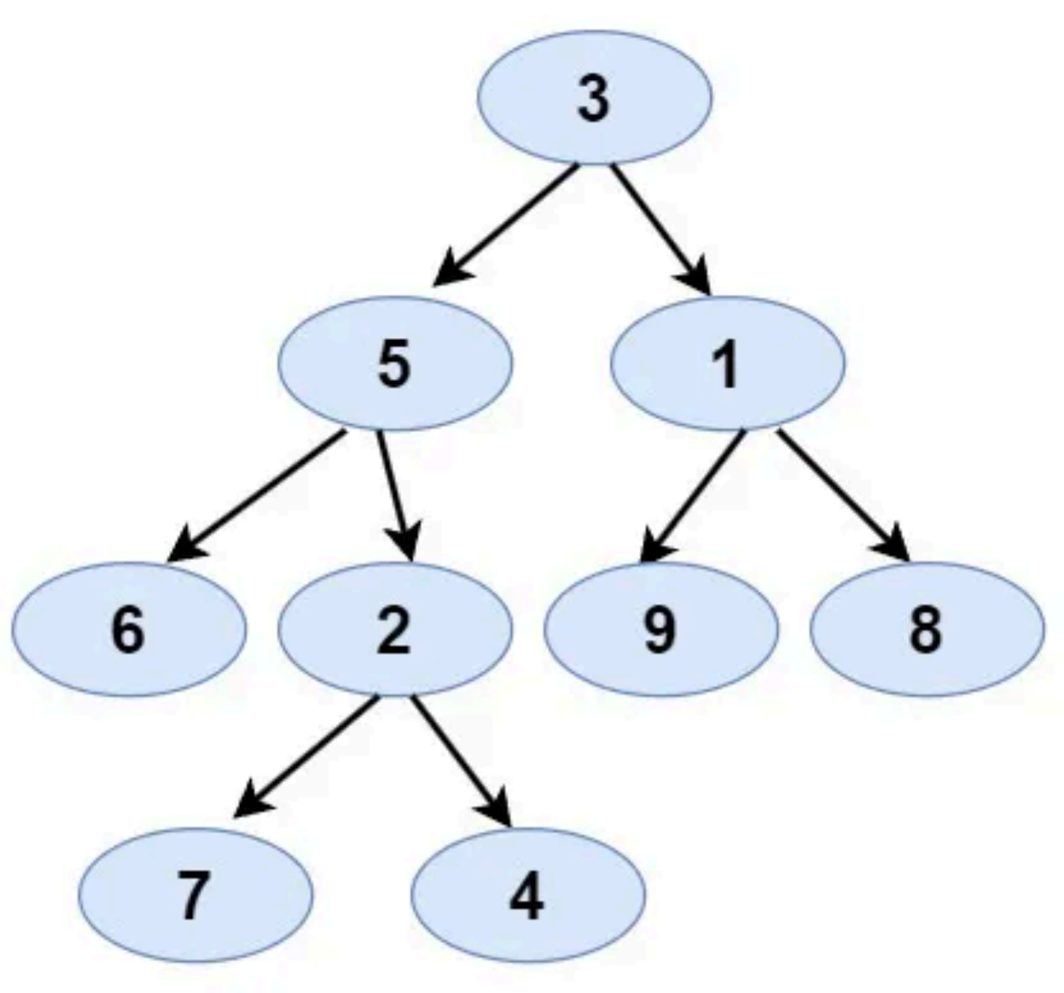
- Input:
  - Preorder: [8,5,9,7,1,12,2,4,11,3]
  - Inorder: [9,5,1,7,2,12,8,4,3,11]
- Expected Output:
  - Tree Representation: [8,5,4,9,7,11,1,12,2,null,3]



- Justification:
  - Start with 8 (from preorder) as the root. Splitting at 8 in inorder, we find the left and right subtrees. Following this pattern recursively, we can construct the tree.

Example 3:

- Input:
  - Preorder: [3,5,6,2,7,4,1,9,8]
  - Inorder: [6,5,7,2,4,3,9,1,8]
- Expected Output:
  - Tree Representation: [3,5,1,6,2,9,8,null,null,7,4]



- Justification:
  - Following the same approach, using 3 as root from preorder, we split the inorder sequence into left and right subtrees and continue recursively.

Solution

Reconstructing a binary tree from its preorder and inorder traversals involves recognizing the structure imposed by these traversal methods. The first element of the preorder traversal always gives us the root of the tree. Once we identify the root, its position in the inorder traversal divides the tree into its left and right subtrees. By harnessing this division recursively for both the left and the right subtrees, we can reconstruct the entire binary tree.

Detailed Steps:

1. **Start with Preorder's Root:** The first element in the preorder list is always the root of the tree.
2. **Find the Root in Inorder:** Once we have the root from preorder, we find its position in the inorder sequence. This position divides the tree into its left and right subtrees.
3. **Recursion:** Based on the root's position in inorder, we can split both the preorder and inorder lists into two halves each - one for the left subtree and the other for the right subtree. We then recursively construct the left and right subtrees.
4. **Base Case:** If the inorder sequence becomes empty, it indicates there's no tree to construct, so we return `null`.

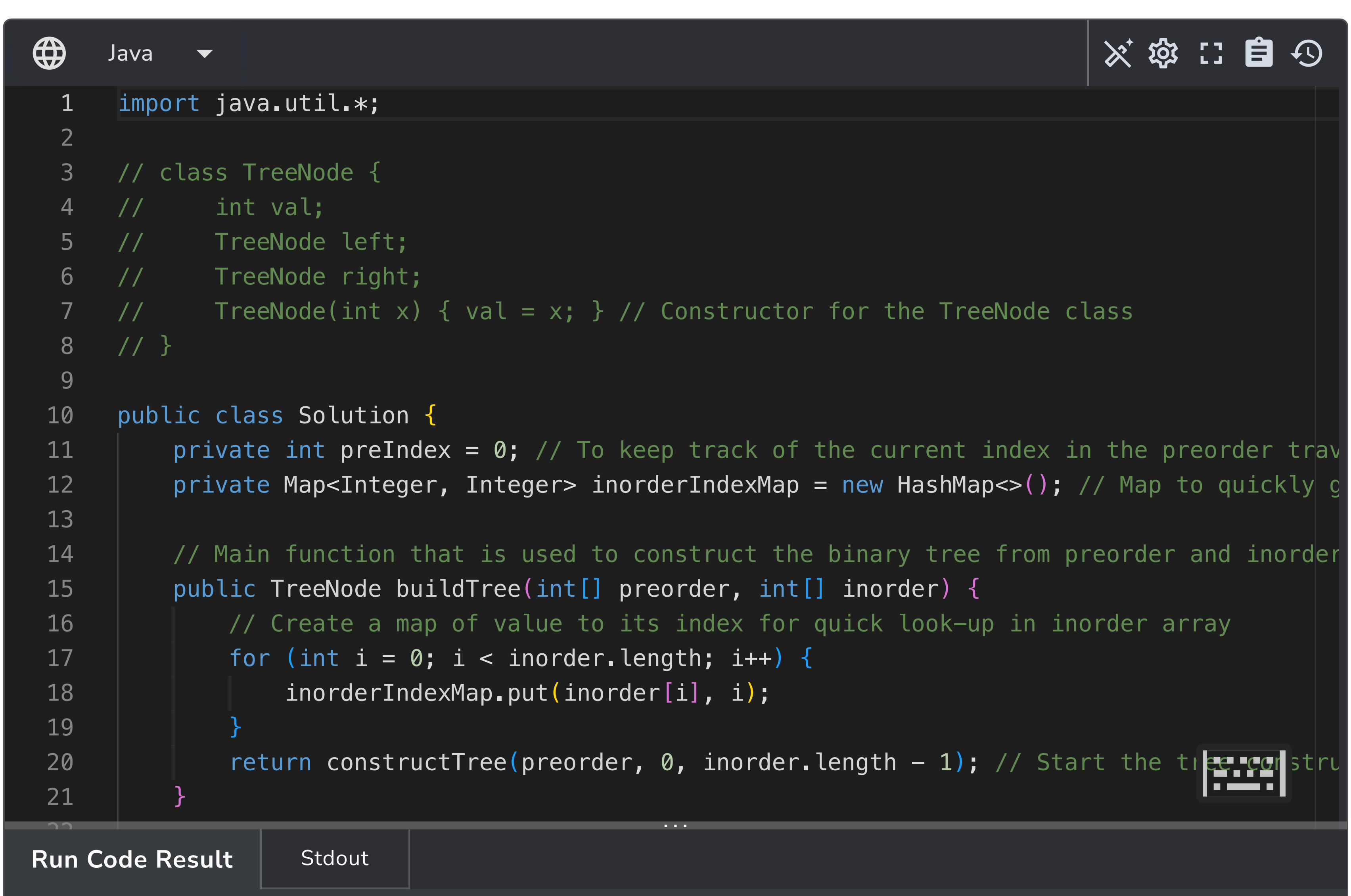
This approach capitalizes on the properties of the preorder and inorder traversals to recursively reconstruct the tree.

Algorithm Walkthrough

Given Preorder: [1,2,4,5,3,6,7] and Inorder: [4,2,5,1,6,3,7]

1. Take the first value from preorder (1) as the root.
2. Find the position of 1 in inorder (position 4).
3. Everything before position 4 in inorder is the left subtree, and everything after is the right subtree.
4. Using the size of the left subtree (3 nodes), we take the next 3 values from preorder [2,4,5] for the left subtree and recursively repeat the process.
5. Similarly, for the right subtree, we use the remaining values from preorder [3,6,7] and follow the same recursive approach.

Code



```java
import java.util.*;

// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; } // Constructor for the TreeNode class
// }

public class Solution {
    private int preIndex = 0; // To keep track of the current index in the preorder trav
    private Map<Integer, Integer> inorderIndexMap = new HashMap<>(); // Map to quickly g

    // Main function that is used to construct the binary tree from preorder and inorder
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        // Create a map of value to its index for quick look-up in inorder array
        for (int i = 0; i < inorder.length; i++) {
            inorderIndexMap.put(inorder[i], i);
        }
        return constructTree(preorder, 0, inorder.length - 1); // Start the tree constru
    }
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n). We are visiting each node once, and the look-up for the inorder index is constant time (due to HashMap or equivalent).
- **Space Complexity**: O(n). The space is majorly used for the hashmap and the recursive stack.

← Previous
Construct Binary Tree From Preorder And Inorder Traversal (Medium)

Next →
Clone Graph (Medium)

☐ Mark as Completed